

COLLEGE OF SCIENCE AND TECHNOLOGY

ITM403: Advanced System Administration

Assignment

Submitted by

Choney Rangdel

Student No.: 02230122

Programme: BE in Information Technology

Date: 12 October 2026

Contents

1. Introduction 1
1.1 Background 1
1.2 Objective 1
1.3 Scope & Environment 1
2. Architecture & Design Structure 2
3. Directories Structuring 2
Milestone 1 3
Milestone 2: Stateful Database 6
Milestone 3: Application Deployment 9
Milestone 4: Ingress 13
Milestone 5 - Autoscaling (HPA + VPA) 17

1. Introduction

1.1 Background

Modern applications are increasingly deployed on container orchestration platforms to achieve portability, resilience, and scalability. Kubernetes has become the de facto standard for this purpose, providing primitives for declarative configuration, self-healing, service discovery, persistent storage, and automated scaling. This assignment explores these primitives through a realistic scenario: deploying a small internal User Management Service on a local Kubernetes cluster.

The service is a REST API that allows staff to create, list, update, and delete user records. Because it is used daily, the system must:

- Persist data across Pod restarts and rescheduling.
- Be reachable at a friendly URL rather than a raw cluster IP.
- Cope with busy periods automatically, without manual intervention.

These requirements map directly onto three Kubernetes concept groups: stateful storage, Ingress-based routing, and autoscaling.

1.2 Objective

The objective of this assignment is to:

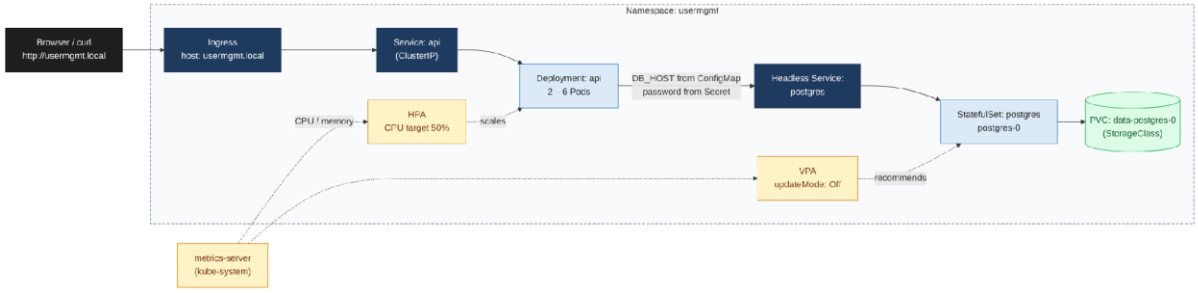
1. Preparing a cluster with the platform add-ons a real application needs; an Ingress controller, a metrics pipeline, and a default StorageClass.
2. Running a database as a StatefulSet with a headless Service and persistent storage, and proving data survives Pod deletion.
3. Packaging a custom application as a versioned container image and running it as a Deployment, using a ConfigMap and a Secret for configuration, probes for health, and resource requests and limits.
4. Exposing the application under a host name using an Ingress resource.
5. Using metrics-server, a HorizontalPodAutoscaler (HPA), and a VerticalPodAutoscaler (VPA) to scale the workload, and explaining why the two autoscalers must not act on the same resource of the same workload.
6. Documenting and defending the design in a written report and a live demonstration.

1.3 Scope & Environment

The deployment is built on a local kind (Kubernetes in Docker) cluster, chosen as the permitted alternative to minikube. All application and database objects are defined as plain YAML manifests and applied with kubectl apply -f.

The system is deployed entirely within the namespace usermgmt-02230122 and is reachable at the host name usermgmt-02230122.local, where 02230122 is the my student ID.

2. Architecture & Design Structure



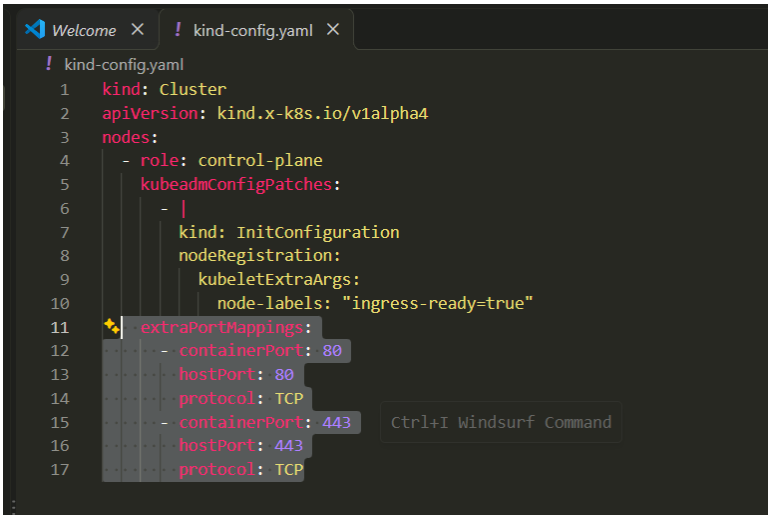
3. Directories Structuring

D:\4Year 1st Sem\ITM403\Assignment\

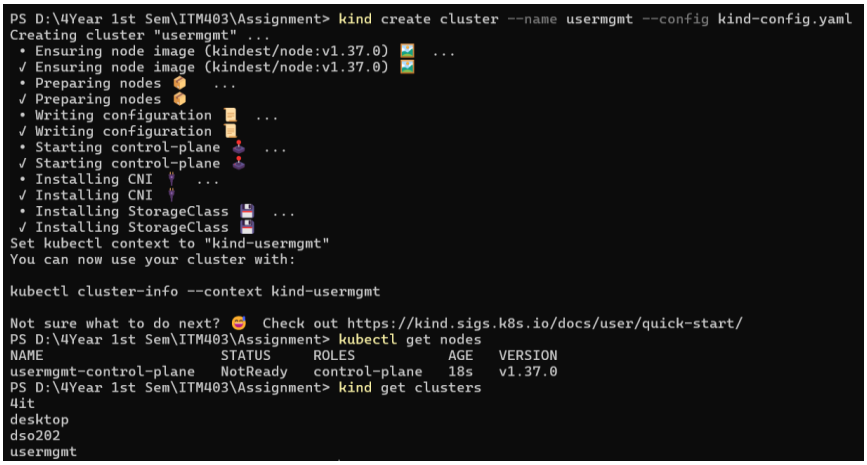
```
│
├── README.md
├── .gitignore
├── kind-config.yaml
├── metrics-server-patch.json
│
├── app\
│   ├── Dockerfile
│   ├── package.json
│   ├── server.js
│   └── .dockerignore
│
├── k8s\
│   ├── 00-namespace.yaml
│   ├── 01-postgres-secret.yaml
│   ├── 02-postgres-headless-svc.yaml
│   ├── 03-postgres-statefulset.yaml
│   ├── 04-api-configmap.yaml
│   ├── 05-api-deployment.yaml
│   ├── 06-api-svc.yaml
│   ├── 06b-api-lb.yaml
│   ├── 07-api-ingress.yaml
│   ├── 08-api-hpa.yaml
│   └── 09-postgres-vpa.yaml
```

Milestone 1

1. Create the cluster config file
   - Create kind-config.yaml in your project root.



2. Create the Cluster



3. Install the metrics-server



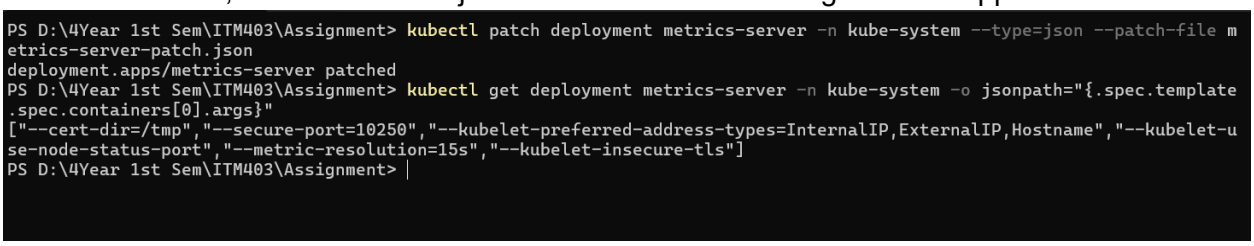
It simply means you're adding the flag --kubelet-insecure-tls to the Metrics Server Deployment so it can talk to kubelets without strict TLS certificate checks.

"path":"/spec/template/spec/containers/0/args"

- I have replaced the whole argos arrays instead of appending. Metrics-server now has only --kubelet-insecure-tls and is missing --cert-dir, --secure-port, -kubelet-preferred-address-types, etc

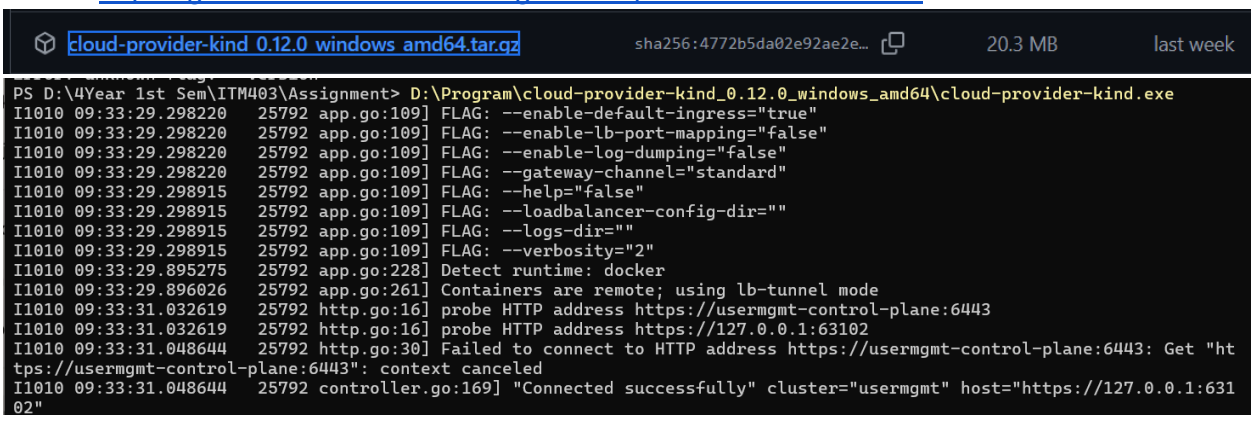


- To fix this, I have created a json file with a full correct args list and applied it.



4. Install the Ingress Controller

- Download the cloud-provider-link from: https://github.com/kubernetes-sigs/cloud-provider-kind/releases



5. Install VPA

```
$ git clone https://github.com/kubernetes/autoscaler.git
Cloning into 'autoscaler'...
remote: Enumerating objects: 252296, done.
remote: Counting objects: 100% (218/218), done.
remote: Compressing objects: 100% (156/156), done.
remote: Total 252296 (delta 134), reused 62 (delta 62), pack-reused 252078 (from 3)
Receiving objects: 100% (252296/252296), 265.28 MiB | 1.06 MiB/s, done.
Resolving deltas: 100% (165053/165053), done.
Updating files: 100% (5471/5471), done.
```

```
Ran@Ran MINGW64 ~
$ cd autoscaler/vertical-pod-autoscaler
./hack/vpa-up.sh
HEAD is now at 238579734 Merge pull request #10339 from adrianmoisey/vpa-release-1.8-update
customresourcedefinition.apiextensions.k8s.io/verticalpodautoscalercheckpoints.autoscaling.k8s.io created
customresourcedefinition.apiextensions.k8s.io/verticalpodautoscalers.autoscaling.k8s.io created
clusterrole.rbac.authorization.k8s.io/system:metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:vpa-actor created
clusterrole.rbac.authorization.k8s.io/system:vpa-status-actor created
clusterrole.rbac.authorization.k8s.io/system:vpa-checkpoint-actor created
clusterrole.rbac.authorization.k8s.io/system:evictioner created
clusterrole.rbac.authorization.k8s.io/system:vpa-updater-in-place created
clusterrolebinding.rbac.authorization.k8s.io/system:vpa-updater-in-place-binding created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-reader created
clusterrolebinding.rbac.authorization.k8s.io/system:vpa-actor created
clusterrolebinding.rbac.authorization.k8s.io/system:vpa-status-actor created
clusterrolebinding.rbac.authorization.k8s.io/system:vpa-checkpoint-actor created
clusterrole.rbac.authorization.k8s.io/system:vpa-target-reader created
clusterrolebinding.rbac.authorization.k8s.io/system:vpa-target-reader-binding created
clusterrolebinding.rbac.authorization.k8s.io/system:vpa-evictioner-binding created
serviceaccount/vpa-admission-controller created
serviceaccount/vpa-recommender created
serviceaccount/vpa-updater created
```

- It uses bash instead of PowerShell because the Vertical Pod Autoscaler install script is written in Linux shell syntax, which PowerShell cannot interpret.

6. Namespace Manifest

```
k8s > ! 00-namespace.yaml
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: usermgmt-02230122
5
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/00-namespace.yaml
namespace/usermgmt-02230122 created
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get ns | Select-String usermgmt

usermgmt-02230122    Active   14s
```

7. Default storage class identified

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get storageclass
NAME                   PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
standard (default)   rancher.io/local-path   Delete          WaitForFirstConsumer   false                  3h38m
PS D:\4Year 1st Sem\ITM403\Assignment>
```

Milestone 2: Stateful Database

In Kubernetes, “stateful” means that each Pod in a StatefulSet keeps a stable identity and persistent data across restarts, unlike stateless Pods which are interchangeable and forget past context.

1. Secrets(Credentials)
   - Store the database name, user, and password in a Kubernetes Secret so no sensitive value appears in the StatefulSet manifest or application code.

k8s > ! 01-postgres-secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: postgres-secret
  namespace: usermgmt-02230122
type: Opaque
stringData:
  POSTGRES_DB: usermgmt
  POSTGRES_USER: usermgmt
  POSTGRES_PASSWORD: cst@123

```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/01-postgres-secret.yaml
secret/postgres-secret created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

2. Headless Service
   - Create a headless Service (clusterIP: None) named postgres so the StatefulSet's Pods get stable DNS names like postgres-0.postgres... instead of a load-balanced virtual IP.

k8s > ! 02-postgres-headless-svc.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: usermgmt-02230122
  labels:
    app: postgres
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - name: postgres
      port: 5432
      targetPort: 5432
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/02-postgres-headless-svc.yaml
service/postgres created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

3. StatefulSet with per-Pod PVC

- What & why: Run PostgreSQL as a StatefulSet (1 replica) so it gets a stable identity (postgres-0) and its own PVC(persistent volume claims) via volumeClaimTemplates, which a Deployment cannot provide safely.

k8s > ! 03-postgres-statefulset.yaml

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: usermgmt-02230122
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
              name: postgres
          envFrom:
            - secretRef:
                name: postgres-secret
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
              subPath: pgdata
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/03-postgres-statefulset.yaml
statefulset.apps/postgres created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get pods -n usermgmt-02230122 -w
NAME         READY   STATUS    RESTARTS   AGE
postgres-0   1/1     Running   0          10s
```

4. Persistence Test

- Inserting a row

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl exec -n usermgmt-02230122 postgres-0 -- psql -U usermgmt -d usermgmt -c "CREATE TABLE IF NOT EXISTS test (id serial PRIMARY KEY, note
 text); INSERT INTO test (note) VALUES ('persistence-check'); SELECT * FROM test;"
WARNING:  database "usermgmt" has no actual collation version, but a version was recorded
CREATE TABLE
INSERT 0 1
 id |        note
----+--------------------
  1 | persistence-check
(1 row)
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl delete pod postgres-0 -n usermgmt-02230122
pod "postgres-0" deleted
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get pods -n usermgmt-02230122 -w
NAME         READY   STATUS    RESTARTS   AGE
postgres-0   1/1     Running   0          4s
```

- Confirming that the row has survived

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl exec -n usermgmt-02230122 postgres-0 -- psql -U usermgmt -d usermgmt -c "SELECT * FROM test;"
WARNING:  database "usermgmt" has no actual collation version, but a version was recorded
 id |        note
----+--------------------
  1 | persistence-check
(1 row)
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get pvc,pv -n usermgmt-02230122
NAME                                   STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data-postgres-0   Bound    pvc-e993fe04-32f8-451b-8a26-379e5710b58c   1Gi        RWO            standard       <unset>                 26m

NAME                                                        CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                                  STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-e993fe04-32f8-451b-8a26-379e5710b58c   1Gi        RWO            Delete           Bound    usermgmt-02230122/data-postgres-0   standard       <unset>                          26m
PS D:\4Year 1st Sem\ITM403\Assignment>
```

- PVC is the request for storage, and PV is the actual storage provided that fulfills that request.

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get statefulset,pod,pvc,pv -n usermgmt-02230122
NAME                                 READY   AGE
statefulset.apps/postgres   1/1     30m

NAME             READY   STATUS    RESTARTS   AGE
pod/postgres-0   1/1     Running   0          5m45s

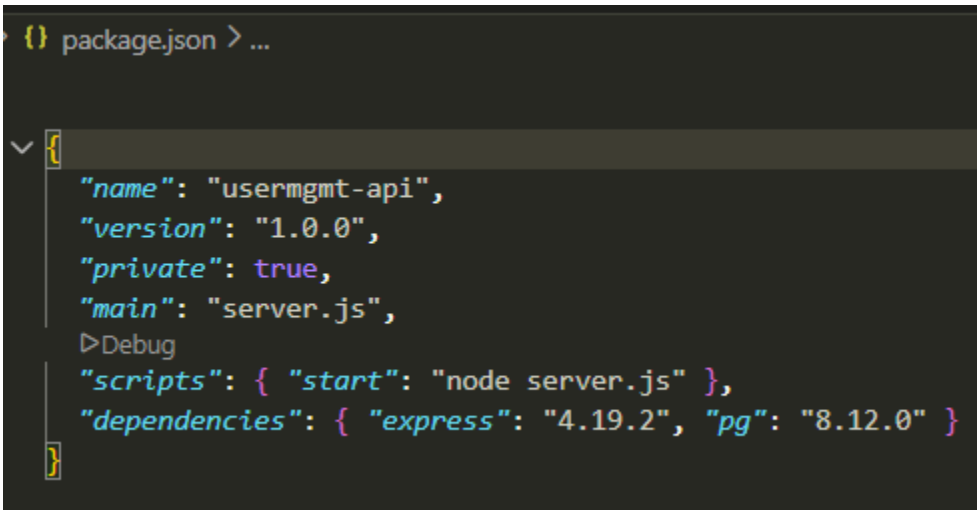
NAME                                   STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/data-postgres-0   Bound    pvc-e993fe04-32f8-451b-8a26-379e5710b58c   1Gi        RWO            standard       <unset>                 30m

NAME                                                        CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                                  STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
persistentvolume/pvc-e993fe04-32f8-451b-8a26-379e5710b58c   1Gi        RWO            Delete           Bound    usermgmt-02230122/data-postgres-0   standard       <unset>                          30m
PS D:\4Year 1st Sem\ITM403\Assignment>
```

Milestone 3: Application Deployment

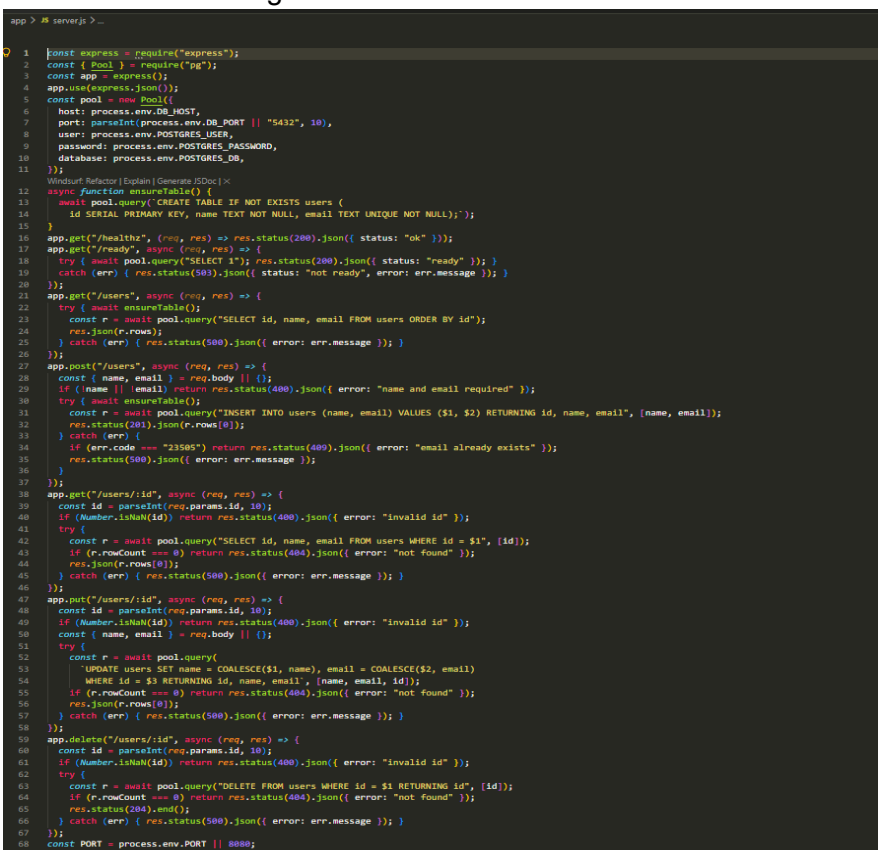
1.Write app/package.json

- Declare the two dependencies (express, pg) and the start script so npm knows how to run the API.



2. Write app/server.js

- Implement the REST API (CRUD + /healthz + /ready) that talks to PostgreSQL using env vars for configuration.



3. Write app/Dockerfile

- Package the app into a small Node alpine image that installs dependencies and runs server.js.

app > Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package.json ./
RUN npm install --omit=dev
COPY server.js ./
EXPOSE 8080
CMD ["node", "server.js"]
```

4. Write app/.dockerignore

- Keep junk like node_modules out of the Docker build context to speed up the build.

app > .dockerignore

```
node_modules
npm-debug.log
.git
Dockerfile
.dockerignore
```

5. Write k8s/04-api-configmap.yaml

- Store non-secret DB connection settings (host, port, database) in a ConfigMap so the image stays config-free.

k8s > ! 04-api-configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: api-config
  namespace: usermgmt-02230122
data:
  DB_HOST: "postgres"
  DB_PORT: "5432"
  DB_NAME: "usermgmt"
```

6. Write k8s/05-api-deployment.yaml

- Define the api Deployment with 2 replicas, probes, resource requests/limits, and env sourced from ConfigMap + Secret.

k8s > ! 05-api-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: usermgmt-02230122
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: usermgmt-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: api-config
            - secretRef:
                name: postgres-secret
          livenessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
```

7. Write k8s/06-api-svc.yaml

- Create a ClusterIP Service named api so other pods and the Ingress can reach the Deployment on a stable name.

k8s > ! 06-api-svc.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: usermgmt-02230122
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 8080
```

8. Build the image

- Turn the app folder into a versioned container image usermgmt-api:1.0 on your host.

```
PS D:\4Year 1st Sem\ITM403\Assignment> docker build -t usermgmt-api:1.0 ./app
[+] Building 99.4s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 181B
 => [internal] load metadata for docker.io/library/node:20-alpine
 => [internal] load .dockerignore
 => => transferring context: 100B
 => [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
 => => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
 => => sha256:b2cbbfe903b08210057809 71ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB
 => => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B
 => => sha256:4feea04c154301db6f4a496efa397b3db96603b1c089c797cfdde77bea8b3287 43.23MB / 43.23MB
 => => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c089c797cfdde77bea8b3287
 => => extracting sha256:b2cbbfe903b08210057809 71ddc5892edcc4ce74c5a48d82e1d2b382edac3122
 => => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096
 => [internal] load build context
 => => transferring context: 3.47kB
 => [2/5] WORKDIR /app
 => [3/5] COPY package.json ./
 => [4/5] RUN npm install --omit=dev
 => [5/5] COPY server.js ./
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:f4fec1465a61d6c077451929cff3404d03d71184b0bf0a9eb493cc5da7a2b55b
 => => exporting config sha256:634131b0e47e0c7acdc2b38124ca74970b5547ebfe602d5854486982ed46c8757
 => => exporting attestation manifest sha256:ef7aaf7ddf9475ea0dc6b7fbe550ab31962bdb7a17b5d110e4d6ee7ddab3a9eb
 => => exporting manifest list sha256:d74f232ee808762cc142068ba137cd55e852d2dc3efdd030e001daf76d46f780
 => => naming to docker.io/library/usermgmt-api:1.0
 => => unpacking to docker.io/library/usermgmt-api:1.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/vd5r3h7uh9le2su8h99ged30v
PS D:\4Year 1st Sem\ITM403\Assignment>
```

9. Load the image into the kind node

- Export the image to a tar and load it via kind load image-archive because Docker Desktop's containerd store breaks kind load docker-image.

```
PS D:\4Year 1st Sem\ITM403\Assignment> docker save usermgmt-api:1.0 -o usermgmt-api.tar
PS D:\4Year 1st Sem\ITM403\Assignment> kind load image-archive usermgmt-api.tar --name usermgmt
PS D:\4Year 1st Sem\ITM403\Assignment> Remove-Item usermgmt-api.tar
PS D:\4Year 1st Sem\ITM403\Assignment>
```

10. Apply the manifests

- Create the ConfigMap, Deployment, and Service in the cluster so Kubernetes can start your api pods

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/04-api-configmap.yaml
configmap/api-config created
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/05-api-deployment.yaml
error: no objects passed to apply
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/06-api-svc.yaml
service/api created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

11. CRUD Demo

Terminal 1 is already running the port-forward on 18080:80. In Terminal 2:

- Creating a user.

```
PS C:\Users\acer> Invoke-RestMethod -Uri "http://localhost:18080/users" -Method Post -ContentType "application/json" -Body '{"name":"Dorji","email":"dorji.cst@rub.edu.com"}'

id name  email
-- ----  -----
 1 Dorji dorji.cst@rub.edu.com
```

- List users

```
PS C:\Users\acer> curl.exe http://localhost:18080/users
[{"id":1,"name":"Dorji","email":"dorji.cst@rub.edu.com"}]
PS C:\Users\acer>
```

- Get one user

```
PS C:\Users\acer> curl.exe http://localhost:18080/users/1
{"id":1,"name":"Dorji","email":"dorji.cst@rub.edu.com"}
PS C:\Users\acer>
```

- Delete the user

```
PS C:\Users\acer> curl.exe -i -X DELETE http://localhost:18080/users/1
HTTP/1.1 204 No Content
X-Powered-By: Express
Date: Sat, 10 Oct 2026 10:47:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5

PS C:\Users\acer> curl.exe http://localhost:18080/users
[]
PS C:\Users\acer>
```

12. Final M3 Evidence

```
PS C:\Users\acer> kubectl get deploy,rs,pods,svc -n usermgmt-02230122
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api   2/2     2            2           19m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/api-795fc9555b      2         2         2       19m

NAME                          READY   STATUS    RESTARTS   AGE
pod/api-795fc9555b-5p48r      1/1     Running   0          19m
pod/api-795fc9555b-9pwcl      1/1     Running   0          19m
pod/postgres-0                1/1     Running   0          107m

NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
service/api          ClusterIP   10.96.226.6    <none>        80/TCP     21m
service/postgres     ClusterIP   None           <none>        5432/TCP   137m
PS C:\Users\acer>
```

Milestone 4: Ingress

1. Write k8s/07-api-ingress.yaml

- Define an Ingress that routes the host usermgmt-02230122.local to the api Service on port 80 and apply it.

k8s > ! 07-api-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-ingress
  namespace: usermgmt-02230122
spec:
  ingressClassName: cloud-provider-kind
  rules:
    - host: usermgmt-02230122.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s/07-api-ingress.yaml
ingress.networking.k8s.io/api-ingress created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

2. Verify the Ingress and its address

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get ingress -n usermgmt-02230122
NAME          CLASS                HOSTS                       ADDRESS   PORTS   AGE
api-ingress   cloud-provider-kind   usermgmt-02230122.local             80      139m
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl describe ingress api-ingress -n usermgmt-02230122
Name:             api-ingress
Labels:           <none>
Namespace:        usermgmt-02230122
Address:
Ingress Class:    cloud-provider-kind
Default backend:  <default>
Rules:
  Host                     Path  Backends
  ----                     ----  --------
  usermgmt-02230122.local
                           /   api:80 (10.244.0.16:8080,10.244.0.15:8080)
Annotations:               <none>
Events:                    <none>
PS D:\4Year 1st Sem\ITM403\Assignment>
```

```
PS C:\Users\acer> kubectl get ingress -n usermgmt-02230122
NAME          CLASS                HOSTS                       ADDRESS                          PORTS   AGE
api-ingress   cloud-provider-kind   usermgmt-02230122.local   172.23.0.8,fc00:f853:ccd:e793::8   80      47h
PS C:\Users\acer>
```

3. Open Notepad as Administrator to edit the hosts file

```
hosts
File   Edit   View   H1   B
#
#      102.54.94.97     rhino.acme.com
#       38.25.63.10     x.acme.com

# localhost name resolution is handled wit
#       127.0.0.1       test.local
#       ::1             localhost
# Added by Docker Desktop
10.2.28.221 host.docker.internal
10.2.28.221 gateway.docker.internal
# To allow the same kube context to work
127.0.0.1 kubernetes.docker.internal
# End of section


## Local - Start ##
::1 localhost #Local Site
127.0.0.1 localhost #Local Site
::1 www.localhost #Local Site
127.0.0.1 www.localhost #Local Site
## Local - End ##

172.23.0.8    usermgmt-02230122.local
```

4. Verify the hostname resolves

```
Pinging usermgmt-02230122.local [172.23.0.8] with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 172.23.0.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

- The ping will time out as kind nodes don't answer ICMP.

- Your Ingress controller (cloud-provider-kind) assigned the Ingress the address 172.23.0.8.
- That IP lives inside Docker Desktop's WSL2 network.
- My Windows host has no route to that subnet. Windows tries to reach it over Wi-Fi and times out.
- Therefore it fails.

5. To solve this problem, I am using using the LoadBalancer service manifest

```
PS D:\4Year 1st Sem\ITM403\Assignment> @'
>> apiVersion: v1
>> kind: Service
>> metadata:
>>   name: api-lb
>>   namespace: usermgmt-02230122
>> spec:
>>   type: LoadBalancer
>>   selector:
>>     app: api
>>   ports:
>>     - port: 80
>>       targetPort: 8080
>> '@ | Set-Content -Path k8s\06b-api-lb.yaml -Encoding utf8
PS D:\4Year 1st Sem\ITM403\Assignment>
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s\06b-api-lb.yaml
service/api-lb created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

```
PS C:\Users\acer> kubectl get svc api-lb -n usermgmt-02230122 -o wide
NAME     TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE   SELECTOR
api-lb   LoadBalancer   10.96.160.42   172.23.0.8    80:32738/TCP   27m   app=api
PS C:\Users\acer>
```

- On my setup, the EXTERNAL-IP shows 172.23.0.8 (a kind internal IP that Windows cannot route to). This is why we fall back to a port-forward in Step 7.

6. Start a port-forward to the LoadBalancer Service

- Tunnel a local host port into the cluster's LoadBalancer Service, because Windows cannot reach the kind pod network directly.

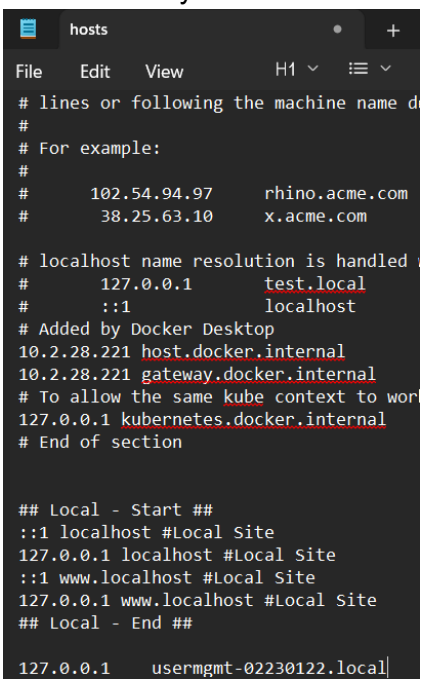
```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl port-forward -n usermgmt-02230122 svc/api-lb 18081:80
Forwarding from 127.0.0.1:18081 -> 8080
Forwarding from [::1]:18081 -> 8080
Handling connection for 18081
```

7. Test the tunnel directly

```
PS C:\Users\acer> curl.exe http://localhost:18081/users
[]
PS C:\Users\acer>
```

8. Update the hosts file

- Tell Windows to resolve usermgmt-02230122.local to 127.0.0.1 so requests to the friendly hostname reach the port-forward.



9. Test the hostname through the tunnel
   - Hit the api at the friendly hostname to prove the Ingress routes by hostname

```
PS C:\Users\acer> curl.exe http://usermgmt-02230122.local:18081/users
[]
PS C:\Users\acer>
```

10. Creating a user through the Ingress

```
PS C:\Users\acer> @'
>> {
>>   "name": "IngressUser",
>>   "email": "ingress@example.com"
>> }
>> '@ | Set-Content body.json -Encoding utf8
PS C:\Users\acer>
PS C:\Users\acer> curl.exe -X POST http://usermgmt-02230122.local:18081/users -H "Content-Type: application/json" -d "@b
ody.json"
{"id":2,"name":"IngressUser","email":"ingress@example.com"}
```

11. Proving that the wrong hostname is rejected

```
PS C:\Users\acer> curl.exe -i http://wrong-host.local:18081/users
curl: (6) Could not resolve host: wrong-host.local
PS C:\Users\acer>
```

12. M4 Explanation:
Ingress is just a way to give your app a friendly hostname instead of a raw IP and port. In my case, the friendly name is usermgmt-02230122.local. When someone types http://usermgmt-02230122.local:18081/users, Windows first looks at my hosts file to figure out which IP that name belongs to. Since I set the hosts file to point usermgmt-02230122.local at 127.0.0.1, which means "my own machine". My port-forward on 18081 is listening on 127.0.0.1, so the request lands there and gets tunnelled into the cluster. Once inside the cluster, the request hits the Ingress controller (cloud-provider-kind). The controller reads the Host: header in the request, which says usermgmt-02230122.local. It looks at my Ingress rule, sees that the host matches, and forwards the request to the api Service. The api Service then load-balances the request across my two api Pods. Each api Pod runs my Express app, which talks to Postgres and returns the list of users as JSON. That's why curl http://usermgmt-02230122.local:18081/users returns my data.

Milestone 5 - Autoscaling (HPA + VPA)

1. Confirming that my metrics are available

```
PS C:\Users\acer> kubectl top pods -n usermgmt-02230122
NAME                   CPU(cores)   MEMORY(bytes)
api-795fc9555b-5p48r   3m           21Mi
api-795fc9555b-9pwcl   3m           41Mi
postgres-0             4m           33Mi
PS C:\Users\acer> kubectl top nodes
NAME                     CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
usermgmt-control-plane   618m         5%       1083Mi          13%
PS C:\Users\acer>
```

2. Write the HPA manifest and apply the HPA.

- Define a HorizontalPodAutoscaler that scales the api Deployment between 2 and 6 replicas when average CPU exceeds 50%.

```
PS D:\4Year 1st Sem\ITM403\Assignment> @'
>> apiVersion: autoscaling/v2
>> kind: HorizontalPodAutoscaler
>> metadata:
>>   name: api-hpa
>>   namespace: usermgmt-02230122
>> spec:
>>   scaleTargetRef:
>>     apiVersion: apps/v1
>>     kind: Deployment
>>     name: api
>>   minReplicas: 2
>>   maxReplicas: 6
>>   metrics:
>>     - type: Resource
>>       resource:
>>         name: cpu
>>         target:
>>           type: Utilization
>>           averageUtilization: 50
>> '@ | Set-Content -Path k8s\08-api-hpa.yaml -Encoding utf8
PS D:\4Year 1st Sem\ITM403\Assignment>
```

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl apply -f k8s\08-api-hpa.yaml
horizontalpodautoscaler.autoscaling/api-hpa created
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get hpa -n usermgmt-02230122
NAME      REFERENCE        TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
api-hpa   Deployment/api   cpu: 1%/50%   2         6         2          5s
PS D:\4Year 1st Sem\ITM403\Assignment>
```

3. Confirm the port-forward for load testing

- We need a way to send load to the api through the Ingress, so to confirm that the tunnel from M4 is still running.

```
PS C:\Users\acer> kubectl port-forward -n usermgmt-02230122 svc/api-lb 18081:80
Forwarding from 127.0.0.1:18081 -> 8080
Forwarding from [::1]:18081 -> 8080
```

4. Install a load-testing tool

- Install hey, a small HTTP load generator, because it lets you send sustained parallel requests to trigger the HPA.

```
PS D:\4Year 1st Sem\ITM403\Assignment> winget install hey
The `msstore` source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function prope
rly (ex. "US").
```

```
The publisher requires that you view the above information and accept the agreements before installing.
Do you agree to the terms?
[Y] Yes  [N] No: y
Starting package install...
  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  100%
Successfully installed
PS D:\4Year 1st Sem\ITM403\Assignment>
```

5. Start a Busybox load pod

- The loop sends requests as fast as possible to the api Service, which should drive CPU above 50% and trigger the HPA.

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl run load-generator --image=busybox:1.36 --restart=Never -n usermgmt-02230
122 -- /bin/sh -c "while true; do wget -q -O- http://api.usermgmt-02230122.svc.cluster.local/users > /dev/null; done"
pod/load-generator created
PS D:\4Year 1st Sem\ITM403\Assignment>
```

6. Watch the HPA scale up

```
pod/load-generator created
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get hpa -n usermgmt-02230122 -w
NAME      REFERENCE         TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
api-hpa   Deployment/api    cpu: 3%/50%   2         6         2          3h42m
api-hpa   Deployment/api    cpu: 37%/50%   2         6         2          3h42m
api-hpa   Deployment/api    cpu: 144%/50%   2         6         2          3h42m
api-hpa   Deployment/api    cpu: 170%/50%   2         6         4          3h42m
api-hpa   Deployment/api    cpu: 137%/50%   2         6         6          3h43m
```

7. Stop the load

- Delete the load pod so CPU drops and the HPA begins scaling back down.

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl delete pod load-generator -n usermgmt-02230122
pod "load-generator" deleted
```

8. Watch the scale-down

- The replicas are also scaling down.

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get hpa -n usermgmt-02230122 -w
NAME      REFERENCE         TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
api-hpa   Deployment/api    cpu: 65%/50%    2         6         6          3h44m
api-hpa   Deployment/api    cpu: 55%/50%    2         6         6          3h44m
api-hpa   Deployment/api    cpu: 63%/50%    2         6         6          3h44m
api-hpa   Deployment/api    cpu: 40%/50%    2         6         6          3h44m
api-hpa   Deployment/api    cpu: 6%/50%     2         6         6          3h44m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         6          3h45m
api-hpa   Deployment/api    cpu: 2%/50%     2         6         6          3h45m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         6          3h45m
api-hpa   Deployment/api    cpu: 2%/50%     2         6         6          3h46m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         6          3h47m
api-hpa   Deployment/api    cpu: 2%/50%     2         6         6          3h47m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         6          3h48m
api-hpa   Deployment/api    cpu: 2%/50%     2         6         6          3h48m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         6          3h48m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         6          3h49m
api-hpa   Deployment/api    cpu: 2%/50%     2         6         5          3h49m
api-hpa   Deployment/api    cpu: 2%/50%     2         6         2          3h49m
api-hpa   Deployment/api    cpu: 1%/50%     2         6         2          3h50m
```

9. Confirming the final state.

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get deploy api -n usermgmt-02230122
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
api    2/2     2            2           2d4h
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get pods -n usermgmt-02230122
NAME                   READY   STATUS    RESTARTS        AGE
api-795fc9555b-5p48r   1/1     Running   2 (5h34m ago)   2d4h
api-795fc9555b-9pwcl   1/1     Running   2 (5h34m ago)   2d4h
postgres-0             1/1     Running   2 (5h34m ago)   2d6h
```

10. Capturing the detailed recommendation bounds

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl describe vpa postgres-vpa -n usermgmt-02230122
Name:         postgres-vpa
Namespace:    usermgmt-02230122
Labels:       <none>
Annotations:  <none>
API Version:  autoscaling.k8s.io/v1
Kind:         VerticalPodAutoscaler
Metadata:
  Creation Timestamp:  2026-10-12T15:25:37Z
  Generation:          1
  Resource Version:    98013
  UID:                 b8809e42-7068-4aef-b697-17838f5dbfd6
Spec:
  Target Ref:
    API Version:  apps/v1
    Kind:         StatefulSet
    Name:         postgres
  Update Policy:
    Update Mode:  Off
Status:
  Conditions:
    Last Transition Time:  2026-10-12T15:26:27Z
    Status:                True
    Type:                  RecommendationProvided
  Observed Generation:     1
  Recommendation:
    Container Recommendations:
```

```
      Container Name:  postgres
      Lower Bound:
        Cpu:     25m
        Memory:  250Mi
      Target:
        Cpu:     25m
        Memory:  250Mi
      Uncapped Target:
        Cpu:     25m
        Memory:  250Mi
      Upper Bound:
        Cpu:     87m
        Memory:  396050260
Events:          <none>
```

11. Capturing the final metrics evidence

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl top pods -n usermgmt-02230122
NAME                   CPU(cores)   MEMORY(bytes)
api-795fc9555b-5p48r   1m           24Mi
api-795fc9555b-9pwcl   4m           61Mi
postgres-0             1m           33Mi
```

12. Capturing the final cluster state

```
PS D:\4Year 1st Sem\ITM403\Assignment> kubectl get hpa,vpa,deploy,pods -n usermgmt-02230122
NAME                                                 REFERENCE         TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/api-hpa          Deployment/api    cpu: 1%/50%   2         6         2          3h58m

NAME                                                        MODE   CPU   MEM     PROVIDED   AGE
verticalpodautoscaler.autoscaling.k8s.io/postgres-vpa       Off    25m   250Mi   True       6m13s

NAME                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/api  2/2     2            2           2d5h

NAME                       READY   STATUS    RESTARTS        AGE
pod/api-795fc9555b-5p48r   1/1     Running   2 (5h41m ago)   2d5h
pod/api-795fc9555b-9pwcl   1/1     Running   2 (5h41m ago)   2d5h
pod/postgres-0             1/1     Running   2 (5h41m ago)   2d6h
PS D:\4Year 1st Sem\ITM403\Assignment>
```

13. HPA/VPA Conflict Explanation

The Horizontal Pod Autoscaler (HPA) works by scaling the number of pods up or down based on CPU usage compared to the resource requests defined in the Deployment. The Vertical Pod Autoscaler (VPA), on the other hand, adjusts those CPU and memory requests themselves. If both were applied to the same Deployment, they would interfere with each other: when the VPA increases requests, the CPU percentage looks lower and the HPA scales down; when the VPA decreases requests, the CPU percentage looks higher and the HPA scales up. This creates a loop where the two controllers keep reacting to each other. To avoid this, the HPA is attached to the stateless API Deployment, which can safely scale in and out, while the VPA is attached to the stateful Postgres StatefulSet in “Off” mode so that it only provides recommendations without restarting the database. This separation ensures stable scaling behavior and avoids conflicts between the two autoscalers.

Conclusion

This assignment deployed a complete user management service on a local kind cluster and demonstrated stateful storage, Ingress routing, and autoscaling. PostgreSQL ran as a StatefulSet with a headless Service, a Secret, and a per-Pod PVC, and a persistence test confirmed data survived Pod deletion. A custom Express API was packaged as a versioned image and deployed as a two-replica Deployment with a ConfigMap, Secret, probes, and resource limits. The API was exposed at usermgmt-02230122.local through an Ingress, with host-based routing demonstrated by rejecting a wrong hostname. The HPA scaled the API from two to six replicas under load and back to two afterward, while the VPA produced CPU and memory recommendations for the database in Off mode. The main difficulty was the Windows and Docker Desktop environment, where the kind pod network was not routable from the host, solved by adding a LoadBalancer Service and tunnelling into it with kubectl port-forward. Overall, each Kubernetes primitive was given a clear role, and the two autoscalers were kept on separate workloads to avoid conflict.